

Problem 1 — Cost-Efficient RAG Application

Background. On a fully managed vector DB, the bill scales with stored vectors (always-on pods), so a large but lightly-queried index becomes a top infra cost. Build a working RAG app on a low-cost store and prove, with numbers, that it is a credible alternative.

Objective. A QA service over a document corpus, backed by a low-cost vector store, with honest evaluation of retrieval quality, answer quality, latency, and cost.

Pick one store. pgvector · Qdrant (self-hosted/embedded) · ChromaDB · LanceDB · FAISS · sqlite-vec. Justify your choice; benchmarking a second is a bonus.

Requirements

1. Ingestion of PDF/HTML/MD; configurable chunk size + overlap (state defaults); idempotent re-ingest (no duplicate vectors).
2. Embed chunks (record model + dimensionality); store vectors + metadata with at least one metadata filter.
3. Top-k retrieval (k is a parameter) → grounded LLM answer that cites the chunks it used; handle 'no relevant context' without hallucinating.
4. Expose as an HTTP endpoint (preferred) or CLI; config via env, no hardcoded secrets; log per-query latency, chunk count, token usage.

Evaluation (the core — use a fixed set of 15–30 questions)

- Retrieval: Recall@k / Hit Rate, MRR, nDCG@k, context precision (needs a notion of which chunk is relevant).
- Answer: faithfulness/groundedness + answer relevance (LLM-as-judge is fine here); EM/F1 if the dataset has gold answers.
- Cost & latency: monthly cost at 100K / 1M / 10M vectors vs a managed DB (state assumptions); p50/p95 retrieval latency; name the trade-offs you accept.

Deliverables

- Runnable RAG service · evaluation harness + results file (all three layers) · cost-comparison table · README with a short Discussion (when would you switch back to managed; was retrieval or generation the weak link).

Scoring (100 pts)

Area	Pts	What we look for
Correctness & ingestion	20	End-to-end works; idempotent; sensible chunking.
Retrieval evaluation	20	Real IR metrics, computed correctly.
Answer evaluation	20	Faithfulness/relevance measured, not asserted.
Cost analysis	20	Credible, assumption-stated comparison across scale.
Engineering & clarity	20	Structure, config hygiene, README, git history.

Problem 2 — LLM-as-Judge Evaluation Pipeline

Background. Teams use a strong model to score/compare outputs when human review can't scale to every change. But judges are themselves biased and noisy — they can prefer the longer answer, the one seen first, or their own model family. Build a judging pipeline and take its biases seriously.

Objective. Given a test case { input, system_prompt, model_output, expected_output?, criteria? }, use an LLM judge to produce a structured quality verdict, with concrete measures to detect and reduce judge bias.

Judging mode. Implement at least one, explain when each fits: pointwise scoring · pairwise A-vs-B · reference-based · reference-free.

Requirements

1. Accept a test suite (JSON/YAML); send a structured judging prompt; parse a STRUCTURED verdict (per-criterion score + rationale + overall), with robust handling of malformed JSON.
2. Define an explicit rubric (e.g. correctness, faithfulness, completeness, instruction-following, tone, safety) — not a bare number.
3. Aggregate per-case → suite report (pass rate, mean scores, win rate); support comparing TWO configs (prompt v1 vs v2, or model A vs B) and declaring a winner.
4. Log every judge prompt + raw response (auditable/replayable); track judge tokens/calls; judge and generator configurable independently.

Bias handling (the heart of this problem) — name it, mitigate it in code, measure it

Bias	Mitigation you must implement / discuss
Position (A/B order)	Run each pair in BOTH orders; average or require agreement; report the flip rate.
Verbosity / length	Control for length; probe with a padded answer; penalize unsupported length in the rubric.
Self-enhancement	Use a judge from a different model family than the generator (or an ensemble).
Sycophancy / style	Force per-criterion grounding; inject a confidently-wrong probe answer.
Score clustering	Calibrate the scale with few-shot anchors, or switch to pairwise.

Validate the judge (at least one)

- Agreement with human/gold labels (agreement rate, Cohen's kappa, or correlation) · test-retest consistency (how often the verdict flips on re-run) · adversarial probe set (verbose-but-wrong, terse-but-correct) and whether it was fooled.

Deliverables

- Runnable pipeline (suite in → report out) · one judging mode + documented rubric · position-bias check with results · a judge-validation artifact · an A/B comparison declaring a winner · README + Discussion (how biased before vs after; would you let it gate a release).

Scoring (100 pts)

Area	Pts	What we look for
Pipeline correctness	20	Suite in, structured verdicts out; robust parsing & logging.
Judging design	20	Sound mode choice + clear structured rubric with rationale.
Bias handling	25	Biases named, mitigated in code, and measured.
Judge validation	20	Agreement/consistency/adversarial evidence, not claims.
Comparison & engineering	15	Clean A/B call; config hygiene, cost tracking, README.